

NumPy

- NumPy is a Python library.
- NumPy is used for working with arrays.
- NumPy is short for "Numerical Python".

What is NumPy?

- NumPy is a Python library used for working with arrays.
- It also has functions for working in the domain of linear algebra, the Fourier transform, and matrices.
- NumPy was created in 2005 by Travis Oliphant. It is an open-source project, and you can use it freely.
- NumPy stands for Numerical Python.

Why Use NumPy?

- In Python, we have lists that serve the purpose of arrays, but they are slow to process.
- NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.
- The array object in NumPy is called `ndarray`; it provides a wide range of supporting functions that make working with `ndarrays` very easy.
- Arrays are frequently used in data science, where speed and resources are crucial.

Why is NumPy Faster Than Lists?

- NumPy arrays are stored in a one continuous place in memory, unlike lists, so processes can access and manipulate them very efficiently.
- This behavior is called locality of reference in computer science.
- This is the main reason why NumPy is faster than lists. Also, it is optimized to work with the latest CPU architectures.

Which Language is NumPy written in?

- NumPy is a Python library and is written partially in Python, but most of the parts that require fast computation are written in C or C++.

alias: In Python, an alias is an alternate name for referring to the same thing.

```
import numpy as np  
arr = np.array([1,2,3,4,5])
```

```

print(arr)
print(type(arr))
print(np.__version__)
arr = np.array((1, 2, 3, 4, 5))#Use a tuple to create a NumPy array:
print(arr)
# 0-D arrays, or Scalars, are the elements in an array.
Each value in an array is a 0-D array.
import numpy as np
arr = np.array(42)
print(arr) #42
arr = np.array([1, 2, 3, 4, 5]) #Create a 1-D array
containing the values 1,2,3,4,5:
print(arr) #[1 2 3 4 5]
arr = np.array([[1, 2, 3], [4, 5, 6]]) #Create a 2-D
array containing two arrays with the values 1,2,3 and
4,5,6:
print(arr)  # [[1 2 3] [4 5 6]]
arr = np.array([[[1,2,4],[5,6,7]], [[1,2,3],[3,4,5]]])

```

Check Number of Dimensions?

```

import numpy as np
a = np.array(42)
b = np.array([1, 2, 3, 4, 5])
c = np.array([[1, 2, 3], [4, 5, 6]])
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4,
5, 6]]])
print(a.ndim)
print(b.ndim)
print(c.ndim)

```

```
print(d.ndim)
```

Higher Dimensional Arrays

```
import numpy as np
arr = np.array([1, 3, 4, 5], ndmin=1)
print(arr)
print(arr.ndim)
```

NumPy Array Indexing

Access Array Elements

- Array indexing is the same as accessing an array element.
- You can access an array element by referring to its index number.
- The indices in NumPy arrays start with 0, meaning that the first element has index 0, and the second has index 1, etc.

```
import numpy as np
arr = np.array([1,3,4,5])
print(arr[0]+arr[1])
```

Access 2-D Arrays

```
import numpy as np
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
print('2nd element on 1st row: ', arr[0, 1])
```

Access 3-D Arrays

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
print(arr[0, 1, 2])
```

Negative Indexing

```
import numpy as np
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
print('Last element from 2nd dim: ', arr[1, -1])
#[1,-1]==[second row,last element]
```

NumPy Array Slicing

- Slicing in Python means taking elements from one given index to another given index.
- We pass a slice instead of an index like this:
`[start: end]`.
- We can also define the step, like this:
`[start:end: step]`.
- If we don't pass start, it's considered 0
- If we don't pass the end, it's considered the length of the array in that dimension
- If we don't pass the step, it's considered 1

Note: The result *includes* the start index, but *excludes* the end index.

Negative Slicing

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[-3:-1])
```

STEP

Use the `step` value to determine the step of the slicing:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[1::2])
```

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[:,2])
```

Slicing 2-D Arrays

```
import numpy as np
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(arr[1, 1:4])
```

```
import numpy as np
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(arr[0:2, 2]) # From both elements, return index 2:
import numpy as np
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(arr[0:2, 1:4]) #From both elements, slice index 1
to index 4 (not included), this will return a 2-D array:
```

NumPy Data Types

Data Types in Python

By default, Python has these data types:

- `strings` - used to represent text data, the text is given under quote marks. e.g. "ABCD"
- `integer` - used to represent integer numbers. e.g. -1, -2, -3
- `float` - used to represent real numbers. e.g. 1.2, 42.42
- `boolean` - used to represent True or False.
- `complex` - used to represent complex numbers. e.g. 1.0 + 2.0j, 1.5 + 2.5j
-

Data Types in NumPy

NumPy has some extra data types, and refer to data types with one character, like `i` for integers, `u` for unsigned integers etc.

Below is a list of all data types in NumPy and the characters used to represent them.

- `i` - integer
- `b` - boolean
- `u` - unsigned integer
- `f` - float
- `c` - complex float
- `m` - timedelta

Checking the Data Type of an Array

- `<U` - Unicode string
- `V` - fixed chunk of memory for other type (void)

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr.dtype) #int64
```

```
import numpy as np
arr = np.array(['apple', 'banana', 'cherry'])
print(arr.dtype) #<U6
```

```
# <= little-endian (not important for strings)
# U = Unicode string
# 6 = maximum length of the strings in the array (in
characters)
```

```
import numpy as np
arr = np.array([1, 2, 3, 4], dtype='S')
print(arr)
print(arr.dtype)
```

```
import numpy as np
arr = np.array([1, 2, 3, 4], dtype='i4')
print(arr)
print(arr.dtype)
```

#A non-integer string like 'a' can not be converted to an integer (will raise an error):

```
import numpy as np
arr = np.array(['a', '2', '3'], dtype='i')
```

Converting Data Type on Existing Arrays

Converting Data Type on Existing Arrays

The best way to change the data type of an existing array, is to make a copy of the array with the `astype()` method.

The `astype()` function creates a copy of the array, and allows you to specify the data type as a parameter.

The data type can be specified using a string, like `'f'` for float, `'i'` for integer etc. or you can use the data type directly like `float` for float and `int` for integer.

```
import numpy as np

arr = np.array([1.1, 2.1, 3.1])
newarr = arr.astype('i')
```

```
print(newarr)
print(newarr.dtype)
print(arr.dtype)
```

```
import numpy as np
arr = np.array([1.1, 2.1, 3.1])
newarr = arr.astype(int)
print(newarr)
print(newarr.dtype)
print(arr.dtype)
```

```
import numpy as np
arr = np.array([1, 0, 3])
newarr = arr.astype(bool)
print(newarr)
print(newarr.dtype) #bool
print(arr.dtype) #int64
```

NumPy Array Copy vs View

The Difference Between Copy and View

- The main difference between a copy and a view of an array is that the copy is a new array, and the view is just a view of the original array.
- The copy *owns* the data, and any changes made to the copy will not affect the original array, and any changes made to the original array will not affect the copy.
- The view *does not own* the data, and any changes made to the view will affect the original array, and any changes made to the original array will affect the view.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
arr[0] = 42
print(arr)
print(x)
```

The copy SHOULD NOT be affected by the changes made to the original array.

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5])
x = arr.view()
arr[0] = 42
print(arr)
print(x)
```

The view SHOULD be affected by the changes made to the original array.

```
import numpy as np
```

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
y = arr.view()
print(x.base) #returns None if the array owns the
data.
print(y.base)
# The copy returns None.
# The view returns the original array.
```

NumPy Array Shape

The shape of an array is the number of elements in each dimension.

Get the Shape of an Array

NumPy arrays have an attribute called `shape` that returns a tuple with each index having the number of corresponding elements.

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
print(arr.shape)
```

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
print(arr.shape) #(2, 4)
```

```
arr = np.array([1, 2, 3, 4], ndmin=5)
print(arr) #[[[[[1 2 3 4]]]]]
print('shape of array :', arr.shape) #shape of array :
(1, 1, 1, 1, 4)
```

The `shape` method returns the shape of an array, i.e., the number of elements in each dimension.

NumPy Array Reshaping

Reshaping arrays

Reshaping means changing the shape of an array.

The shape of an array is the number of elements in each dimension.

By reshaping, we can add or remove dimensions or change the number of elements in each dimension.

Reshape From 1-D to 2-D

- Convert the following 1-D array with 12 elements into a 2-D array.
- The outermost dimension will have 4 arrays, each with 3 elements:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(4, 3)
print(newarr)
```

Reshape From 1-D to 3-D

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(2, 3, 2)
print(newarr)
```

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(2, 2, 3)
print(newarr)
```

Example

Try converting 1D array with 8 elements to a 2D array with 3 elements in each dimension (will raise an error):

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
```

Check if the returned array is a copy or a view:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
print(arr.reshape(2, 4).base)
```

Unknown Dimension

You are allowed to have one "unknown" dimension.

Meaning that you do not have to specify an exact number for one of the dimensions in the reshape method.

Pass `-1` as the value, and NumPy will calculate this number for you.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
newarr = arr.reshape(2, 2, -1)
print(newarr)
```

Note: We can not pass `-1` to more than one dimension.

Flattening the arrays

Flattening an array means converting a multidimensional array into a 1D array.

We can use `reshape(-1)` to do this.

#Convert the array into a 1D array:

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
newarr = arr.reshape(-1)
print(newarr)
```

NumPy Array Iterating

```
import numpy as np
```

```

arr = np.array([1, 2, 3])
for x in arr:
    print(x)

import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
for x in arr: #In a 2-D array it will go through all
the rows.
    print(x)

```

If we iterate on a n -D array it will go through $n-1$ th dimension one by one.

```

import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
for x in arr:
    for y in x:
        print(y)

import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10,
11, 12]]])
for x in arr:
    print("x represents the 2-D array:")
    print(x)

```

```

import numpy as np

```

```

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10,
11, 12]]])
for x in arr:
    print("x represents the 2-D array:")
    print(x)

for x in arr:
    for y in x:
        for z in y:
            print(z)

```

Iterating Arrays Using nditer()

The function `nditer()` is a helping function that can be used from very basic to very advanced iterations. It solves some basic issues which we face in iteration, let's go through it with examples.

Iterating on Each Scalar Element

In basic `for` loops, iterating through each scalar of an array we need to use n `for` loops which can be difficult to write for arrays with very high dimensionality.

```

import numpy as np
arr = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
for x in np.nditer(arr):
    print(x)

```

```

arr = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

```

```
for x in np.nditer(arr):  
    print(x)
```

Iterating an Array With Different Data Types

We can use the `op_dtypes` argument and pass it the expected data type to change the data type of elements while iterating.

NumPy does not change the data type of an element in-place (where the element is in an array), so it requires additional space to perform this action. That extra space is called a buffer, and in order to enable it in `nditer()`, we pass `flags ['buffered']`.

```
import numpy as np  
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])  
for x in np.nditer(arr, flags=['buffered'],  
op_dtypes=['S']):  
    print(x)
```

Iterating With Different Step Size

We can use filtering followed by iteration.

```
import numpy as np  
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
```

```
for x in np.nditer(arr[:, ::2]):  
    print(x)
```

The `enumerate()` function in Python is a built-in function that adds a counter to an iterable (such as a list, tuple, string, or set) and returns it as an enumerate object.

```
import numpy as np  
arr = np.array([1, 2, 3])  
for idx, x in np.ndenumerate(arr):  
    print(idx, x)  
    print(type(idx))    # type of the index  
    print(type(x))      # type of the value
```

```
import numpy as np  
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])  
for idx, x in np.ndenumerate(arr):  
    print(idx, x)
```

NumPy Joining Arrays

Joining means putting contents of two or more arrays in a single array.

In SQL we join tables based on a key, whereas in NumPy we join arrays by axes.

We pass a sequence of arrays that we want to join to the `concatenate()` function, along with the axis. If axis is not explicitly passed, it is taken as 0.

```
import numpy as np
a = np.array([1,2,3,4])
b = np.array([5,6,7,8])
arr = np.concatenate((a,b),axis=0) # axis = 0; means
vertically
print(arr)

arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[5, 6], [7, 8]])
arr = np.concatenate((arr1, arr2), axis=1) # axis == 1
means horizontally
print(arr)
```

Joining Arrays Using Stack Functions

Stacking is the same as concatenation; the only difference is that stacking is done along a new axis.

We can concatenate two 1-D arrays along the second axis, which would result in putting them one over the other, ie, stacking.

We pass a sequence of arrays that we want to join to the `stack()` method along with the axis. If the axis is not explicitly passed, it is taken as 0.

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.stack((arr1, arr2), axis=1)
print(arr)
```

Stacking Along Rows

NumPy provides a helper function: `hstack()` to stack along rows.

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.hstack((arr1, arr2))
print(arr)
```

Stacking Along Columns

NumPy provides a helper function, `vstack()`, to stack along columns.

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.vstack((arr1, arr2))
print(arr)
```

Stacking Along Height (depth)

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.dstack((arr1, arr2))
print(arr)
```


NumPy Splitting Array

Splitting NumPy Arrays

Splitting is the reverse operation of Joining.

Joining merges multiple arrays into one, and splitting breaks one array into multiple.

We use `array_split()` for splitting arrays. We pass the array we want to split and the number of splits.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6])
newarr = np.array_split(arr, 3)
print(newarr)
```

Note: The return value is a list containing three arrays.

If the array has fewer elements than required, it will adjust from the end accordingly.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6])
newarr = np.array_split(arr, 4)
```

```
print(newarr)
```

Note: We also have the method `split()` available, but it will not adjust the elements when elements are less in source array for splitting like in the example above, `array_split()` worked properly, but `split()` would fail.

```
# Split the 2-D array into three 2-D arrays.
```

```
import numpy as np
arr = np.array([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12]])
newarr = np.array_split(arr, 3)
print(newarr)
```

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6])
newarr = np.array_split(arr, 3)
print(newarr)
print(newarr[0])
print(newarr[1])
print(newarr[2])
```

```
import numpy as np
arr = np.array([[1, 2], [3, 4], [5, 6], [7, 8],
               [9, 10], [11, 12]])
newarr = np.array_split(arr, 3)
print(newarr)
```

Split the 2-D array into three 2-D arrays along columns.

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10,
               11, 12], [13, 14, 15], [16, 17, 18]])
newarr = np.array_split(arr, 3, axis=1)
print(newarr)
```

An alternate solution is using
hsplit() **opposite of hstack()**

```
import numpy as np
```

```
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12], [13, 14, 15], [16, 17, 18]])
newarr = np.hsplit(arr, 3)
print(newarr)
```

Note: Similar alternates to `vstack()` and `dstack()` are available as `vsplit()` and `dsplit()`.

NumPy Searching Arrays

```
#Find the indexes where the value is 4:
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 4, 4])
x = np.where(arr == 4)
print(x) #The example above will return a
tuple: (array([3, 5, 6],))
```

```
# Find the indexes where the values are even:
```

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
x = np.where(arr%2 == 0)
print(x)
```

Search Sorted

Find the indexes where the values are even:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
x = np.where(arr%2 == 0)
print(x)
```

The `searchsorted()` method is assumed to be used on sorted arrays.

```
import numpy as np
arr = np.array([6, 7, 8, 9])
x = np.searchsorted(arr, 7)
print(x)
```

Example explained: The number 7 should be inserted on index 1 to remain the sort order.

The method starts the search from the left and returns the first index where the number 7 is no longer larger than the next value.

```
# Find the indexes where the value 7 should be
inserted, starting from the right:
import numpy as np
arr = np.array([6, 7, 8, 9])
x = np.searchsorted(arr, 7, side='right')
print(x)
```

Multiple Values

To search for more than one value, use an array with the specified values.

```
import numpy as np
arr = np.array([1, 3, 5, 7])
x = np.searchsorted(arr, [2, 4, 6])
print(x)
print(type(x))
```

NumPy Sorting Arrays

Sorting means putting elements in an *ordered sequence*.

Ordered sequence is any sequence that has an order corresponding to elements, like numeric or alphabetical, ascending or descending.

The NumPy ndarray object has a function called `sort()`, that will sort a specified array.

```
import numpy as np
arr = np.array([3, 2, 0, 1])
print(np.sort(arr))
```

Note: This method returns a copy of the array, leaving the original array unchanged.

```
import numpy as np
arr = np.array(['banana', 'cherry', 'apple'])
print(np.sort(arr))
```

```
# Sort a boolean array:
```

```
import numpy as np
arr = np.array([True, False, True])
print(np.sort(arr))
```

Sorting a 2-D Array

If you use the `sort()` method on a 2-D array, both arrays will be sorted:

```
import numpy as np
arr = np.array([[3, 2, 4], [5, 0, 1]])
print(np.sort(arr))
```

NumPy Filter Array

Filtering Arrays

Getting some elements out of an existing array and creating a new array out of them is called *filtering*.

In NumPy, you filter an array using a *boolean index list*.

A boolean index list is a list of booleans corresponding to indices in the array.

```
import numpy as np
arr = np.array([41, 42, 43, 44])
x = [True, False, True, False]
newarr = arr[x]
print(newarr)
```

Creating the Filter Array

```
import numpy as np
arr = np.array([41, 42, 43, 44])
filter_arr = []
for element in arr:
    if element > 42:
        filter_arr.append(True)
    else:
        filter_arr.append(False)
newarr = arr[filter_arr]
print(filter_arr)
print(newarr)
```

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
filter_arr = arr % 2 == 0
newarr = arr[filter_arr]
print(filter_arr)
print(newarr)
```

Random Numbers in NumPy

What is a Random Number?

Random number does NOT mean a different number every time. Random means something that can not be predicted logically.

```
from numpy import random
x = random.randint(100)
print(x)
x = random.rand() # it will give 0-1 value
print(x)
```

```
x = random.randint(100,size=(5))
print(x) # it will give a 5-size random array
x = random.randint(100,size=(5,5)) #Generate a
2-D array with 3 rows, each row containing 5
random numbers:
print(x) # it will create 5x5 matrix
x = random.rand(5)
print(x)
x = random.rand(5,5)
print(x)
```

Generate a Random Number From an Array

The `choice()` method allows you to generate a random value based on an array of values.

The `choice()` method takes an array as a parameter and randomly returns one of the values.

```
x = random.choice([1, 3, 4, 5, 20, 20, 89, 89])  
print(x)  
x = random.choice([3, 5, 7, 9], size=(3, 5))  
print(x)
```

Random Data Distribution

What is Data Distribution?

Data Distribution is a list of all possible values and how often each value occurs.

Such lists are important when working with statistics and data science.

The random module offers methods that returns randomly generated data distributions.

Random Distribution

A random distribution is a set of random numbers that follow a certain *probability density function*.

Random Distribution

A random distribution is a set of random numbers that follow a certain *probability density function*.

Probability Density Function: A function that describes a continuous probability. i.e., probability of all values in an array.

```
# now we will generate 1-D with 100 values where each
value has to be 3,5,7,9
# the probability for the value 3 is set to be 0.1
# the probability for the value 5 is set to be 0.3
# the probability for the value 7 is set to be 0.6
# the probability for the value 9 is set to be 0
# The sum of all probability numbers should be "1"

from numpy import random
x = random.choice([3, 5, 7, 9], p=[0.1, 0.3,
0.6, 0.0], size=(100))
print(x)
```

```
from numpy import random
x = random.choice([3, 5, 7, 9], p=[0.1, 0.3, 0.6,
0.0], size=(3, 5))
```

```
print(x)
```

Random Permutations

Random Permutations of Elements:

A permutation refers to an arrangement of elements. e.g. [3, 2, 1] is a permutation of [1, 2, 3] and vice-versa.

The NumPy Random module provides two methods for this: `shuffle()` and `permutation()`.

Shuffling Arrays

Shuffling means changing the arrangement of elements in place. i.e., in the array itself.

```
from numpy import random
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
random.shuffle(arr)
```

```
print(arr)
```

The `shuffle()` method makes changes to the original array.

Generating Permutations of Arrays

Generate a random permutation of the elements of the following array:

```
from numpy import random
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(random.permutation(arr))
```

The `permutation()` method *returns* a rearranged array (and leaves the original array unchanged).

Visualize Distributions With Seaborn

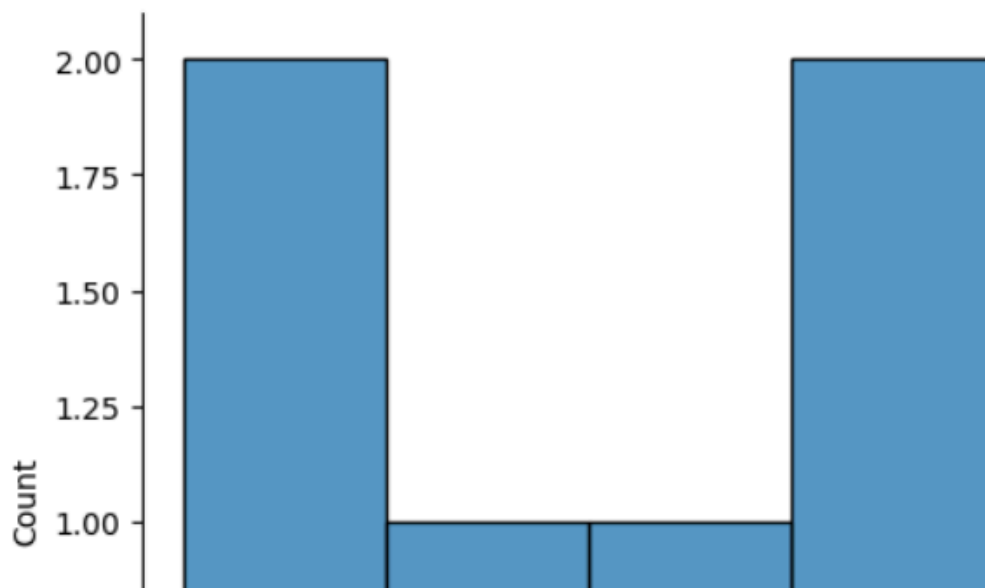
Install Seaborn.

If you have [Python](#) and [PIP](#) already installed on a system, install it using this command:

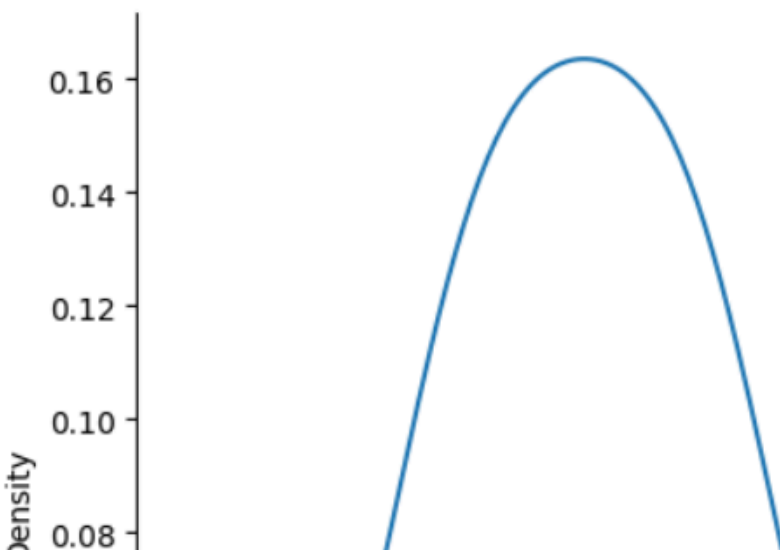
Displots

Displot stands for distribution plot; it takes as input an array and plots a curve corresponding to the distribution of points in the array.

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.displot([0, 1, 2, 3, 4, 5])
plt.show()
```




```
import matplotlib.pyplot as plt
import seaborn as sns
sns.displot([0, 1, 2, 3, 4, 5], kind="kde")
plt.show()
```

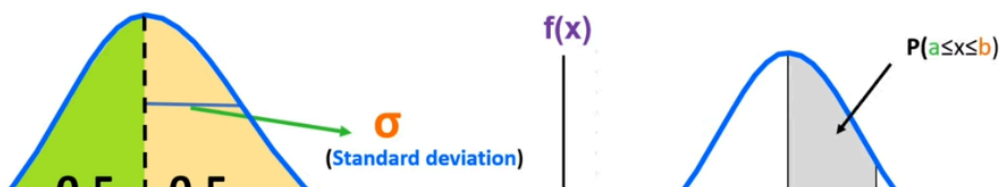


Normal (Gaussian) Distribution



Introduction to Normal Distribution

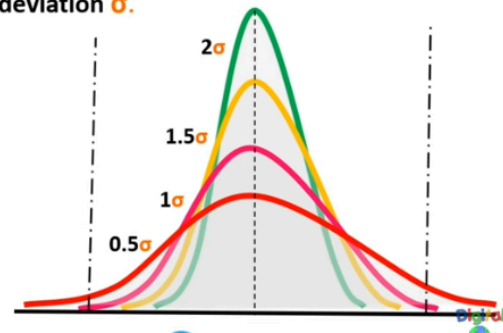
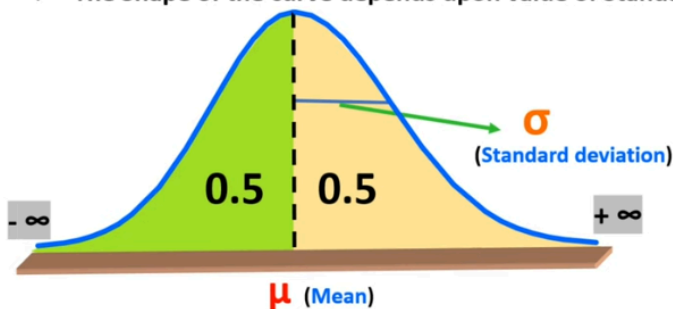
- ❖ Normal Distribution is a **Symmetric, Bell-shaped** distribution.
- ❖ Symmetrical about the mean (μ). Centerline represent the **mean** of distribution.
- ❖ Extends from $-\infty$ to $+\infty$.
- ❖ Property of normal distribution is that area under the curve.
- ❖ Total area under the curve is **1**. Half of this curve is **0.5**
- ❖ Shaded **area under the curve** is the probability that X is between **a** and **b**





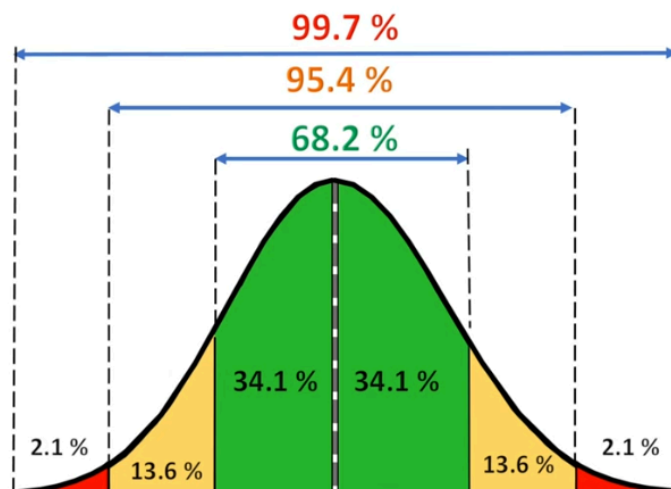
Introduction to Normal Distribution

- ❖ Normal Distribution is a **Symmetric, Bell-shaped** distribution.
- ❖ Symmetrical about the mean (μ). Centerline represent the **mean** of distribution.
- ❖ Extends from $-\infty$ to $+\infty$.
- ❖ Property of normal distribution is that area under the curve.
- ❖ Total area under the curve is **1**. Half of this curve is **0.5**
- ❖ Shaded **area under the curve** is the probability that X is between **a** and **b**
- ❖ The Shape of the curve depends upon value of Standard deviation σ .



Introduction to Normal Distribution

3 Sigma σ Rule
OR
Empirical Rule
OR
68-95-99.7 rule





Introduction to Normal Distribution

- ❖ Normal Distribution curve can be transformed to **Standard normal distribution**.

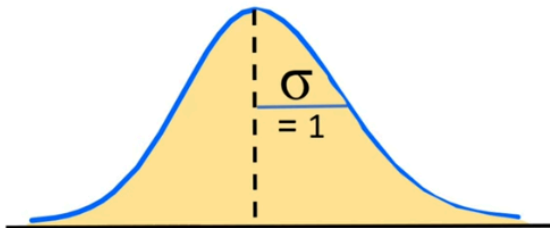
$$Z = \frac{X - \mu}{\sigma}$$

X = random variable

σ = Standard deviation

μ = Mean

Z = Number of standard deviations from x to the mean



Normal Distribution

The Normal Distribution is one of the most important distributions.

It is also called the Gaussian Distribution after the German mathematician Carl Friedrich Gauss.

It fits the probability distribution of many events, eg. IQ Scores, Heartbeat etc.

Use the `random.normal()` method to get a Normal Data Distribution.

It has three parameters:

`loc` - (Mean) where the peak of the bell exists.

`scale` - (Standard Deviation) how flat the graph distribution should be.

`size` - The shape of the returned array.

```
from numpy import random
x = random.normal(size=(2, 3))
print(x)
```

Python Virtual Environment

```
def myfunc(n): ?????????????????????????????????????????????????????????????
    return abs(n - 50)
thislist = [100, 50, 65, 82, 23]
thislist.sort(key = myfunc)
print(thislist)
```

Not Bitwise Operator ????????????????

**Lambda Function with map, filter, and
reduce???????????**

PIP and Virtual Environment ????????????????????????